

Aufgabe 7 — Lagerverwaltung

Objektorientierte Programmierung

Das Projekt

Gegeben ist die Struktur eines Lagers. Ein **ItemBatch** beschreibt einen Posten eines Produkts mit Produktname, Stückzahl, Preis und Lagerzone. Die Lagerzone wird als **enum StorageZone** modelliert: COLD, DRY, HAZARDOUS, BULK. Die abstrakte Klasse **BaseBatch** enthält gemeinsame Felder und Validierungslogik. Ein **Warehouse** verwaltet Bestände, berechnet Werte und verarbeitet Entnahmen.

Sie dürfen beliebige Hilfsmethoden hinzufügen. Vorgegebene Signaturen dürfen nicht verändert werden.

Aufgabe 1 — Posten implementieren

$0.5 + 1 + 0.5 + 0.5 + 0.5 + 0.5 = 3.5$ Punkte

Das Interface **org.warehouse.ItemBatch** und die abstrakte Klasse **BaseBatch** sind bereits vorgegeben. **BaseBatch** enthält die Felder `productName`, `amount`, `unitPrice`, `batchNumber` sowie die Validierungslogik.

- a) Implementieren Sie **org.warehouse.StoredBatch**, die von **BaseBatch** erbt. Ergänzen Sie das Feld `zone` (`StorageZone`) und einen Konstruktor in der Reihenfolge `productName`, `amount`, `unitPrice`, `batchNumber`, `zone`. Rufen Sie `super(...)` korrekt auf.
- b) Die **IllegalArgumentException** wird bereits von **BaseBatch** geworfen wenn `productName` leer ist, `amount < 0` oder `unitPrice < 0`.
- c) Das Enum **StorageZone** ist bereits vorgegeben. Stellen Sie sicher, dass die Zone korrekt gespeichert und über `zone()` zurückgegeben wird.

Aufgabe 2 — Lager vervollständigen

$0.5 + 1 + 1.5 + 1.5 + 3 + 2.5 = 10$ Punkte

Die Klasse **org.warehouse.SimpleWarehouse** verwaltet beliebig viele Posten.

- a) Sorgen Sie dafür, dass **SimpleWarehouse** das Interface **Warehouse** implementiert und nach dem Konstruktor leer ist.
- b) Implementieren Sie **addBatch**, sodass ein Posten hinzugefügt wird.
- c) Implementieren Sie **totalValue**, die für alle Posten `amount * unitPrice` aufsummiert.
- d) Implementieren Sie **totalAmountOf(String productName)**, die die Gesamtstückzahl eines Produkts über alle Posten zurückliefert. Gibt es kein solches Produkt, soll 0 zurückgegeben werden.
- e) Implementieren Sie **removeItems(String productName, int amount)**, die Stückzahlen eines Produkts entnimmt. Die Entnahme soll Posten in Einfügereihenfolge leeren. Falls insgesamt nicht genug Bestand vorhanden ist, soll eine **IllegalArgumentException** geworfen werden und der Zustand unverändert bleiben. Posten mit `amount == 0` nach der Entnahme sollen entfernt werden.
- f) Implementieren Sie **valueByZone()**, die eine **EnumMap<StorageZone, Double>** zurückliefert, welche für jede Lagerzone den Gesamtwert (`amount * unitPrice`) aller Posten dieser Zone enthält.

Punkteverteilung

Aufgabe 1: 3.5 Punkte

Aufgabe 2: 10 Punkte

Gesamt: 13.5 Punkte